

GEP Intern – Technology: Interview Playbook

How R1 (technical) and R2 (technical / functional + HR) actually go — questions + answers

Ojas Srivastava | B.Tech AI, SVNIT Surat | Submitted resume freeze | Hyderabad / Mumbai

Built from GEP campus reports (GFG, LinkedIn intern offers, AmbitionBox, GEP Campus Connect) + your resume. Not a generic DSA dump.

0. How the loop really works

GEP campus tech hiring is consistent across intern and SDE reports: **OA** → **Technical 1** → **Technical 2** → **HR** (sometimes folded into R2). Campus drives often finish in 1–2 days. Interviewers **do not use a fixed question paper**. They take your intro + resume and drill whatever you said. Overstatement is the main reject reason.

R1 (~45–60 min) Intro → OOP / CS fundamentals → 1 Easy–Medium DSA (arrays, hashing, graphs/trees) → one project deep-dive → sometimes a puzzle.

R2 (~35–50 min) Deeper on **the same projects** + more CS / a second coding question (Fibonacci in a range, reverse array) + “design this using OOP” + AI/LLM if you mentioned it. Functional: can you explain LogiFlow like a product.

HR (~10–20 min) Why GEP, family, location (Hyd and Mumbai — stay open), strengths/weakness, any questions. Not a formality if you sound uninterested or rigid on location.

Pattern from people who got intern/SDE offers: foundations > hard CP; project ownership > buzzwords; think-aloud > silent coding. A GEP intern (Shivam Kumar, campus SDE 2025): one Easy–Medium DSA per technical round (arrays/graphs) + long project discussion; questions came from *his* intro and code, not a bank. A GEP intern from SVNIT (Sarthak Chauhan): R1 OOP pillars + project auth; R2 favourite DS (trees) then BFS-style problem + puzzle; HR hobbies + *can you relocate*.

Your drive (from PPT): two interview rounds after OA. Treat R1 as fundamentals + LogiFlow. Treat R2 as Community Hero / Career Automation / Nexus + HR. IFFCO is on the resume — they can still ask even if you skip it in the intro. **AirHelp is not on the submitted resume** — old cheat sheets still mention it; do not volunteer it.

1. Locked intro (speak this, then stop)

Good afternoon sir, ma'am. I am Ojas Srivastava. I am a pre-final year B.Tech student in Artificial Intelligence at SVNIT Surat, with a CGPA of 9.20.

My coursework includes Object-Oriented Programming, Database Management Systems, Operating Systems, Computer Networks, and Design and Analysis of Algorithms.

I was part of the team that secured Global rank 106 in Google Solution Challenge 2026. We built LogiFlow, a supply-chain routing system. I also built Community Hero for Google Vibe2Ship 2026, Global Top 20 — a civic issue reporting PWA.

My coding workflow is built around Cursor, with MCP and plugins. I still write the spec and review every merge.

Currently I am Chairperson of Nexus, SVNIT — the departmental cell of CSE and AI. We manage and mentor 500-plus students.

I treat that the same way I treat a project: I own the outcome. GEP works on AI-powered procurement and supply chain. That is the kind of work I have already started on — taking operational data to a decision, and owning it till it ships.

Do not say in R1: Intern–Technology title, Hyderabad/Mumbai, IFFCO (unless they open the resume), MCP product names, LangGraph, Azure, Angular, “main lead of the whole team.”

Do say if asked: technical lead on railway / comparator / hybrid pipelines; ranking on time, cost, delay risk — not “type of goods” unless you can defend it from the code.

2. Minute-by-minute: Technical R1

1. **0–2 min** — **camera, resume, “tell me about yourself.”** Speak the locked intro. Stop. Smile. Do not add IFFCO or location.

2. **2–12 min** — **they pick a word from the intro.** Most likely LogiFlow or OOP from coursework. Have a 60-second LogiFlow story ready (section 4).

3. **12–22 min** — **CS fundamentals.** OOP 10 minutes is extremely common. Then OS or DBMS if time.

4. **22–40 min** — **one DSA.** Clarify → brute → better → code → dry-run → complexity. Never silent.

5. **40–50 min** — **puzzle or “any questions?”** Ask one technical question (section 9). Do not ask stipend here.

3. Minute-by-minute: Technical / functional R2 + HR

R2 is not a second copy of R1. Reports: they go **deeper on one project** (ML/AI if you have it — you do: Gemini on Community Hero; delay ranking on LogiFlow), then a **simpler coding** question, then culture.

1. **0–3 min** — **intro again?** 30-second version: name, LogiFlow Top 106, Community Hero Top 20, Nexus chairperson, GEP close. Do not repeat full coursework.

2. **3–20 min** — **functional:** “Explain LogiFlow to a product manager.” Time, cost, delay. Who uses it. What breaks.

3. **20–35 min** — **AI they heard in PPT:** What is an LLM vs a rules engine? What is a guardrail? Do *not* invent LangGraph.

4. **35–45 min — second code or OOP design** (bank / cart / Zoomcar-style).

5. **HR block:** why GEP, location **both**, family, weakness, joining date. Stay open to Hyderabad and Mumbai.

Intern-length warning: one on-campus intern R2 was ~10 min (OOP purpose, DS vs algorithms, a lateral-thinking web Q, “where in 5 years”). AI-intern campuses have used names **Preliminary** then **Functional** instead of Tech 1 / Tech 2 — same content. Do not panic if R2 is short; still finish cleanly.

Official: process aims to close in 1–2 days. Fail a campus process → **6-month cooling** before you can sit GEP again. Offices they hire campus for: **Mumbai (Airoli / CJB)** and **Hyderabad (Madhapur)**. Not Noida.

4. Project questions (this is where they actually decide)

LogiFlow — they will start here

Q: Tell me about LogiFlow.

What people usually say: “It is an AI logistics app we made for a hackathon, we used Python and it got Top 106.”

You say: We ranked logistics corridors on time, cost, and delay risk so an operator can pick a route under constraints. Google Solution Challenge Global Top 106. I was technical lead on the railway pipeline, the comparator pipeline, and the hybrid pipeline. Data: 15,000+ train-day records, 580+ corridors. FastAPI + Next.js UI. Redis for hot reads around 100–400 ms. pytest 100/100 on pricing/routing rules. Ranking is the product; ML delay signal is one input, not the whole app.

Q: What was *your* code vs the team’s?

What people usually say: “I did a bit of everything / I was the main lead of the whole product.”

You say: Co-lead. I owned railway path: geometry/reads, ranking into the API, tests. Comparator/hybrid: compare modes so the user sees trade-offs, not one dump. Frontend was shared; I do not claim I wrote every React file.

Q: Why Redis? What if Redis is down?

What people usually say: “Redis is fast. We used it for speed.”

You say: Corridor metadata is read far more than written. Cache-aside: miss → SQL → SET with TTL. If Redis is down, FastAPI still answers from SQL, slower. Redis is not the source of truth. Stale reads inside TTL are OK for maps; not OK for an irreversible commit.

Q: How do you rank time vs cost vs delay?

What people usually say: “We used AI / a neural network.”

You say: Multi-objective. Pareto-style: do not hide a slow cheap route. Delay risk came from a gradient-boosting model on 15k+ train-days — I can say MAE around 22 minutes if asked, and that it is sklearn-style boosting, not PyTorch. If I am unsure of a number I will not invent it.

Q: This is procurement. How is routing relevant?

What people usually say: “GEP and LogiFlow are the same product.”

You say: GEP is buy + move + plan. LogiFlow is the **move** layer: given constraints, which path. I have not built a purchase-order system. I have built decisioning on operational logistics data.

Community Hero

Q: What is Community Hero?

What people usually say: “A social app with Gemini.”

You say: Civic issue reporting PWA. Vibe2Ship Global Top 20. I solo-owned React/Express, Firestore, Cloud Run. Citizen drops a geo-tagged report; Gemini categorizes; admin reviews. Guard checks: empty/noisy input never hits the model blindly; bad model output does not break routing — fallback to manual review.

Q: What if Gemini is wrong or down?

What people usually say: “Then the app fails / we retry.”

You say: Treat the model as untrusted. Validate input. If output is not a known category, store raw text and flag for admin. Prefer a missed auto-tag over a wrong auto-route.

Career Automation Stack (on the resume — they can open GitHub)

Q: What is this stack? The GitHub only has YAML.

What people usually say: “I built 14k questions and emailed 10k recruiters with AI.”

You say: Personal scheduled pipelines, not a GEP product. Internship Scout: ingest → dedup → HTML digest over 966 companies, GitHub Actions. Hiring Scout: public-source contacts, rate-limited SMTP — I do not spam. OA Forge: 14k+ evidence-tracked questions, 5.5k+ tests, hybrid C++ judge with g++ and subprocess limits. The public repo is **workflow YAML only** so personal data stays private. I can walk a job graph: checkout, Python, secrets, cache, commit state. Application code is private; I still own the design.

Q: Is scraping / cold email legal? Why should GEP trust you?

What people usually say: “Everyone does it / it is just automation.”

You say: Personal learning scope: public pages, rate limits, no purchased lists. At GEP I follow company policy and legal. The engineering I would transfer is scheduling, retries, idempotent state, and tests — the same discipline as a procurement job that must not double-send.

Q: How does the C++ judge work?

You say: User code → compile with g++ (C++17, -O2) → run under time/memory limits in a subprocess → compare stdout to expected. If compile fails, verdict is compile error, not WA. I do not claim a kernel sandbox like Codeforces.

Cursor / MCP (only if they bite)

Q: What is MCP? Did AI write your resume projects?

What people usually say: “I use all the newest AI tools, I have 20 plugins.”

You say: Cursor is my IDE. MCP lets the agent call tools I already use — I will not list product names unless asked. I write the spec, I review every diff, I merge. Same as a junior who types fast: I still own the code. 28k+ Python LOC and 446 commits in that workflow is volume, not a claim that the agent is the author of record.

Nexus

Q: Chairperson — what do you actually own?

What people usually say: “I manage 500–600 students daily.”

You say: Departmental technical cell of CSE and AI. I own planning and delivery of workshops, contests, mentoring — 500-plus students across events, not a daily headcount. Journey if asked: Member → Social Media Coordinator → Academic Mentor → Technical Representative → Chairperson, about two years. GEP said they like ownership. This is the people version of the same habit.

IFFCO (on resume — they may open it even if intro skipped it)

Q: What did you do at IFFCO?

What people usually say: “Auto authentication and professional deadlines.”

You say: Jun–Jul 2025, technical team. Node.js, Express, MySQL. 10+ REST APIs with validation and error handling. Tools that automated 50+ daily plant workflows. Git, Docker, CI/CD. First time production: logs and stack traces when a flow broke, then a fix, then tell the mentor.

5. OOP — 10 minutes (almost every GEP R1)

Q: Four pillars, with an example from your code.

What people usually say: Definitions copied from GFG, no example.

You say: **Encapsulation:** hide MySQL password/connection inside a module; handlers only see `createReport()`. **Abstraction:** FastAPI route is “get ranked corridors,” not the SQL+Redis steps. **Inheritance:** a base `Pipeline` with `run()` / rail vs hybrid override. **Polymorphism:** comparator and railway both expose rank, different internals. Access: public API, private helpers.

Q: Abstract class vs interface.

What people usually say: “Same thing.”

You say: Abstract class: shared code + some unimplemented methods. Interface/protocol: only the contract. Python: ABC vs Protocol. I use a shared base when pipelines share validation; a protocol when only the `rank()` shape is shared.

Q: Design a bank / cart with OOP.

What people usually say: Draw random boxes.

You say: Classes: Customer, Account, Transaction. Account has `withdraw()` that checks balance (encapsulation). Savings/Current inherit Account. Transfer is a Transaction. For “last item in cart”: inventory row with version/quantity; `UPDATE ... WHERE qty > 0`; if 0 rows updated, tell the user. That is optimistic concurrency, not hope.

6. OS + DBMS + SQL (they fail people here)

Q: Process vs thread. Deadlock.

What people usually say: One-line mix-up.

You say: Process = program in execution, own memory. Thread = lighter, shares process memory. Deadlock needs all four: mutual exclusion, hold and wait, no preemption, circular wait. Mutex = one owner. Semaphore = counter.

Q: ACID. Index. JOIN.

What people usually say: Expand the acronym only.

You say: Atomicity all-or-nothing; Consistency rules; Isolation concurrent txs; Durability after commit. Index: faster lookup, slower writes. INNER JOIN: matches both sides. LEFT JOIN: all left + NULL if no match. GROUP BY then HAVING for aggregates. WHERE cannot filter COUNT(*).

Q: Write SQL: departments with more than 5 employees.

You say: `SELECT dept, COUNT(*) FROM emp GROUP BY dept HAVING COUNT(*) > 5`

Q: SQL vs NoSQL — you have MySQL and Firestore.

You say: LogiFlow/IFFCO: joins, tariffs, validation — SQL. Community Hero: documents per report, geo, fast iterate — Firestore. Not “SQL is old.”

Q: MySQL vs MongoDB (TalentBattle GEP). You listed Firestore, not Mongo.

You say: I have not used MongoDB in a shipped project. Closest: Firestore documents. SQL when I need joins and constraints. Document store when the record is a blob with nested fields and the schema is still moving.

Q: Python vs Java.

You say: Python: fast to ship APIs, data, tests (LogiFlow, scouts). Java: typed, JVM, common in enterprise. I have Java in coursework. GEP FTE often uses C# / Angular — I have not faked those on the resume; I can learn the product stack.

Q: Stack vs queue vs list vs tree.

You say: Stack LIFO (undo, DFS, call stack). Queue FIFO (BFS, job queue). List: $O(1)$ head insert, $O(n)$ index. Tree: hierarchy; BST left < root < right. Graph: corridors in LogiFlow — nodes + edges, BFS for unweighted hops.

7. DSA they actually give (Easy–Medium)

Coding loop every time: constraints, example, brute, better, code, dry-run, $O(\text{time})/O(\text{space})$.

Q: Reverse an array / string.

You say: Two pointers i, j ; swap until they meet. $O(n)$ time, $O(1)$ extra.

Q: First non-repeating character / remove all repeating characters (GEP intern R1, quoted).

You say: Count in a map (or array[256]). First unique: second pass, first count==1. Remove repeats: keep chars with count==1, or use two pointers + map. Say an example: `swiss` → first unique `w`.

Q: Two Sum / frequency map.

You say: Hash map. Two Sum: store complement. $O(n)$.

Q: Middle of a linked list.

You say: Slow/fast pointers. Fast moves 2, slow moves 1. When fast hits end, slow is mid. $O(n)$ time, $O(1)$ extra.

Q: Alt+Tab OA (quoted on GFG Associate SD): $n = 4$, list $\{1, 2, 3, 4\}$, $k = 3 \rightarrow \{3, 1, 2, 4\}$. Catch $k \geq n$.

You say: The k -th window comes to front. Index $i = (k - 1) \bmod n$; rotate that element to index 0, shift the prefix right. Dry-run $k = 3$ and $k = 7$ before coding.

Q: Seconds between two clock times (GEP intern OA, easy).

You say: Parse HH:MM:SS to seconds since midnight; subtract; if negative, add 24h. Watch midnight wrap.

Q: Kadane — max subarray. Dry-run `[-2, 1, -3, 4, -1, 2, 1, -5, 4]`.

You say: Running sum; if sum < 0 reset to 0; track max. Answer 6, subarray `[4, -1, 2, 1]`.

Q: Binary search.

You say: `lo=0, hi=n-1; while lo<=hi: mid=(lo+hi)//2`. Sorted only. $O(\log n)$.

Q: Fibonacci in a range (GFG GEP R2).

You say: Iterate $a, b = 0, 1$; print while in range. Two methods: loop and formula if they want. Watch overflow.

Q: BFS / trees (Sarathak's GEP intern R2).

You say: Favourite DS if asked: **hash map** for APIs or **graph/queue** for corridors — pick one and stay. BFS: queue, level-order, shortest unweighted path. Level-order: push root; pop; push left then right.

Q: Primes 1 to 100.

You say: Trial division or sieve. Sieve: mark multiples. Do not say “I used AI to generate sieve” — write it.

8. Puzzles (GFG GEP R2)

Q: 12 balls, 1 different weight, 3 weighings.

You say: Ternary split 4–4. Balance tells which group. Repeat. Classic.

Q: 3L and 5L jugs, make 4L.

You say: Fill 5, pour into 3 (2 left in 5), empty 3, pour 2 into 3, fill 5, pour into 3 (1 space) → 4 in the 5L.

If stuck: state assumptions, try a small case, do not freeze.

9. AI they may ask (PPT: wide idea, not research)

Q: AI vs ML vs DL. Supervised vs unsupervised.

You say: AI broad; ML learns from data; DL neural nets. Supervised = labels (classify/regress). Unsupervised = clusters. Linear regression predicts a number, not a class.

Q: How would you put an LLM on procurement data?

What people usually say: “LangGraph agents, RAG, fine-tune GPT.”

You say: I have not used LangGraph. Pattern I have used: ingest → validate → model (Gemini categorize) → guard → human/admin if unsure. Same shape on invoices/POs: extract fields, never auto-post a bad total. I would learn their workflow tool on the job.

Q: Hadoop on OA (seen on some AI-intern / SE papers).

You say: Distributed processing of large files on a cluster. Map then reduce. I have **not** run a Hadoop job. I will not pretend. SQL/Python pipelines are what I have shipped.

Q: Purpose of OOP / DS vs algorithms (short intern R2).

You say: OOP: bundle data+behavior so a CorridorRanker can change internals without breaking callers. DS = how we store (hash, tree). Algorithms = how we process (BFS, Kadane). You pick a DS to make the algorithm cheap.

Q: How do you know a website's language without reading the code?

You say: Look at `lang` on HTML, `Content-Language` header, or script URLs. If they want CS: detect from tokens/stopwords.

I say I am guessing and ask which layer they mean — HTTP, HTML, or NLP.

10. HR — typical questions and answers

HR is short. People fail by lying on location, badmouthing college, or having no reason for GEP except stipend.

Q: What do you know about GEP? (intern HR, quoted)

What people usually say: Recite Wikipedia or name random modules.

You say: Software + consulting + managed services for procurement and supply chain. Products I will name at a high level: SMART (source-to-pay), NEXXE (supply chain), Quantum Intelligence as the AI layer. I will not fake screens. Tech interviews almost never quiz S2P cycles — that is the consulting track.

Q: Why GEP?

What people usually say: “Great brand, good package, work-life balance.”

You say: AI-powered procurement and supply chain — software plus how enterprises actually buy and move. LogiFlow is routing decisions on operational data. I want that class of problem at company scale, with ownership.

Q: Where do you see yourself in 5 years? (intern R2/HR)

You say: Engineer who can own a service on their procurement/supply-chain stack — APIs, data, tests — and who can add AI behind guards. Not “CTO.”

Q: Hyderabad or Mumbai?

What people usually say: Pick one then flip later / “wherever, I don’t care.”

You say: I am open to both. No family constraint. I will go where the team is. (Do not invent a favourite.)

Q: Why software if your degree is AI?

What people usually say: “AI is just a name / I want to switch.”

You say: I ship systems — APIs, data, tests. AI is a component (ranking, Gemini) on top of that, with guards. GEP needs both.

Q: Strength / weakness.

What people usually say: “Perfectionist.”

You say: Strength: I finish — design to deploy, I own it (Nexus + Community Hero). Weakness: I can over-build a personal tool. I time-box and ship a thin version first, then iterate — that is how I keep scout pipelines from never releasing.

Q: Family background / where are you from?

You say: Short, factual. Prayagraj roots, college in Surat. No drama.

Q: Do you have offers? Will you join?

You say: If true: this is the process I am in; if I get GEP I intend to join. Do not play companies against them.

Q: Questions for us?

What people usually say: “No” or “What is the stipend?”

You say: How do interns contribute to AI / agent workflows on procurement data in the first month? What does a good intern look like at 8 weeks?

11. Weak vs strong — same question

Topic	Weak (common)	Strong (you)
Intro	3 minutes, IFFCO story, location, 7 MCP names	Locked 50s intro, then stop
LogiFlow	“AI app, I did everything”	Rank time/cost/delay; three pipelines; tests
Gemini	“We used AI for intelligence”	Guard + fallback; model is untrusted
DSA	Jump to code, silent	Clarify, brute, optimize, dry-run
Don’t know	Guess LangGraph / Azure	“I have not used X; closest is Y”
HR location	Only Mumbai / only Hyd	Both, no constraint

12. Day-of rules

- Print the submitted resume. Do not explain a project that is not on it (AirHelp: only if they ask GitHub).
- Every noun you say is a follow-up: Redis, pytest, Firestore, Gemini, MCP, Chairperson.
- If coding: talk. Partial working brute force > silent optimal.
- R1: no stipend, no location speech. R2/HR: location both.
- End each round: “Thank you. I enjoyed this.” Then one question if they offer time.

13. Sources (so you know this is not invented)

- GEP Campus Connect India: 2-day close, Hyd + Mumbai, intern for 2028 passout — gep.com/careers/join-us/campus-connect
- GFG: GEP SDE — OA (aptitude, maths, OS/DBMS, 1 coding); R1 OOP+resume DS+project+array+puzzle; R2 project

ML + Fibonacci in range; HR family + location

- GFG: GEP SDE-1 on-campus — R2 OOP bank design + last-item cart + 12 balls + jugs
- LinkedIn: Shivam Kumar (GEP intern → SDE 2025) — 2 tech + HR; 1 Easy–Medium DSA/round; questions from intro/resume
- LinkedIn: Sarthak Chauhan (SVNIT, GEP intern) — OOP R1; trees+BFS R2; HR relocate
- AmbitionBox: “Explain key projects you implemented”; GEP SWE stack often C# / Angular / SQL for FTE — you do not fake that; you say open to learn
- GFG intern on-campus: first unique char; difficulties in project; short R2; everyone who reached HR got intern
- TalentBattle ASE 2023: OOP, access specifiers, stack/queue/list/tree, mid of LL, BS, primes 1–100, MySQL vs Mongo
- UnsaidTalks: Maanya Jain AI intern 2024 (Preliminary + Functional + HR; Hadoop on OA); Sanjay Gupta (intern Mumbai, FTE Hyd or Mumbai)
- Official cooling period 6 months on fail. Campus offices: Mumbai CJB and Hyderabad only.
- Your PPT: two interviews; ownership; wide AI; supply chain. Your submitted PDF is the only source of truth in the room.

Golden rule: If it is not on the resume and you cannot dry-run it, do not say it. Clarify → approach → code → dry-run → complexity. Never silent.